Digital Comprehensive Summaries of Uppsala Dissertations
from the Faculty of Science and Technology 2599

Advances in Speculative Side Channel Mitigations and New Exploitations

PAVLOS AIMONIOTIS

ACTA UNIVERSITATIS
UPSALIENSIS
2025

ISSN 1651-6214
ISBN 978-91-513-2619-1
urn:nbn:se:uu:diva-566433

UPPSALA
UNIVERSITET

Dissertation presented at Uppsala University to be publicly examined in Polhemsalen, Ångströmlaboratoriet, Lägerhyddsvägen 1, Uppsala, Friday, 12 December 2025 at 09:15 for the degree of Doctor of Philosophy. The examination will be conducted in English. Faculty examiner: Charles N. Millican Professor Yan Solihin (University of Central Florida).

Abstract
Aimoniotis, P. 2025. Advances in Speculative Side Channel Mitigations and New Exploitations. *Digital Comprehensive Summaries of Uppsala Dissertations from the Faculty of Science and Technology* 2599. 48 pp. Uppsala: Acta Universitatis Upsaliensis. ISBN 978-91-513-2619-1.

Speculative execution optimizes processor performance by predicting the correct path of execution and executing instructions before it is legal to do so, introducing potential vulnerabilities when the prediction falls short and the program follows the wrong path of execution. While the processor recovers the architectural state, the microarchitectural state remains and can be exploited. In these vulnerabilities, that leverage the speculative side-channel (known as Spectre attacks), a secret is improperly accessed speculatively and then leaked by passing it to a transmitter instruction that leaves irreversible changes in the microarchitecture. Several proposed defenses try to close this security hole by prohibiting the propagation of speculatively loaded values, or by delaying the transmitter instructions (e.g., loads) from executing under speculation. Each of these proposed solutions leads to considerable performance degradation and, in certain instances, fails to entirely mitigate the security vulnerability.

This thesis tries to address the challenges of performance and security by proposing three works on optimizing the performance of existing speculative side-channel defenses, namely Janus, ReCon, and Doppelganger Loads, and two works on addressing security vulnerabilities by demonstrating attack scenarios, namely DOIN!, and Reorder Buffer Contention.

Regarding enhancing the performance of existing speculative mechanisms, Janus applies two of the simplest defense ideas in the same processor design and uses reinforcement learning to select the optimal performance-wise underlying defense mechanism on the fly, ReCon leverages non-speculative information leakage in an efficient manner to enable the execution of speculative load instructions that access non-secret values, and Doppelganger Loads employs an address predictor to unlock more speculative memory-level parallelism by predicting speculative load addresses in a secret-independent way. All the aforementioned techniques are able to recover a significant part of the lost performance introduced by speculative side-channel defenses.

On the other hand, to expose security vulnerabilities, DOIN! attack exploits inclusive caches and the co-existence of instructions and data in last-level caches to create observable timing differences in the data cache, surpassing speculative defenses, and Reorder Buffer Contention adds secret-dependent contention to the reorder buffer pushing in or out, on demand, a transmitter instruction able to leak the value of the speculatively loaded secret.

Keywords: computer architecture, speculative execution, processor, security, out-of-order execution, side-channels, microarchitecture

Pavlos Aimoniotis, Department of Information Technology, Computer Systems, Box 337, Uppsala University, SE-75105 Uppsala, Sweden.

© Pavlos Aimoniotis 2025

ISSN 1651-6214
ISBN 978-91-513-2619-1
URN urn:nbn:se:uu:diva-566433 (http://urn.kb.se/resolve?urn=urn:nbn:se:uu:diva-566433)

List of papers

This thesis is based on the following papers, which are referred to in the text by their Roman numerals.

I Aimoniotis, Pavlos, and Stefanos Kaxiras. "JANUS: A Simple and Efficient Speculative Defense using Reinforcement Learning." In 2024 IEEE 36th International Symposium on Computer Architecture and High Performance Computing (SBAC-PAD), pp. 25-36. IEEE, 2024.

II Aimoniotis, Pavlos, Amund Bergland Kvalsvik, Xiaoyue Chen, Magnus Själander, and Stefanos Kaxiras. "ReCon: Efficient Detection, Management, and Use of Non-Speculative Information Leakage." In Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO), pp. 828-842. 2023.
HiPEAC Paper Award

III Kvalsvik, Amund Bergland, Pavlos Aimoniotis, Stefanos Kaxiras, and Magnus Själander. "Doppelganger Loads: A Safe, Complexity-Effective Optimization for Secure Speculation Schemes." In Proceedings of the 50th Annual International Symposium on Computer Architecture (ISCA), pp. 1-13. 2023.
HiPEAC Paper Award

IV Aimoniotis, Pavlos, Amund Bergland Kvalsvik, Magnus Själander, and Stefanos Kaxiras. "Data-Out Instruction-In (DOIN!): Leveraging Inclusive Caches to Attack Speculative Delay Schemes." In 2022 IEEE International Symposium on Secure and Private Execution Environment Design (SEED), pp. 49-60. IEEE, 2022.

V Aimoniotis, Pavlos, Christos Sakalis, Magnus Själander, and Stefanos Kaxiras. "Reorder Buffer Contention: A forward Speculative Interference Attack for Speculation Invariant Instructions." IEEE Computer Architecture Letters (CAL) 20, no. 2 (2021): 162-165.
Best of CAL Award

Reprints were made with permission from the publishers.

Other publications

These papers were produced as part of the author's PhD studies but are not part of this thesis.

Aligholipour, Rashid, Pavlos Aimoniotis, Stefanos Kaxiras, and Yuan Yao. "RXT: RefleXive Address Translation for Pointer-Chasing Workloads." In 2025 IEEE International Parallel and Distributed Processing Symposium (IPDPS), pp. 1062-1073. IEEE, 2025.

Chen, Xiaoyue, Pavlos Aimoniotis, and Stefanos Kaxiras. "How addresses are made." In 2023 IEEE International Symposium on Workload Characterization (IISWC), pp. 223-225. IEEE, 2023.

Chen, Xiaoyue, Pavlos Aimoniotis, and Stefanos Kaxiras. "Clueless: A tool characterising values leaking as addresses." In Proceedings of the 11th International Workshop on Hardware and Architectural Support for Security and Privacy (HASP), pp. 27-34. 2022.

Contents

I Introduction 8

II Background and Related Work 12
- II.1 Speculative Execution 12
- II.2 Cache side-channels 13
- II.3 Speculative Side-Channel Attacks 14
  - II.3.1 Spectre Attacks 14
- II.4 Speculative Defenses 16
  - II.4.1 Invisible Speculation 16
  - II.4.2 Speculative Delay Schemes 16
  - II.4.3 Enhancing Speculative Defenses 20
- II.5 Attacks on Speculative Defenses 22

III Novel Speculative Defense Tactics for Enhanced Performance 24
- III.1 Paper I - Janus: A Simple and Efficient Speculative Defense using Reinforcement Learning 24
- III.2 Paper II - ReCon: Efficient Detection, Management, and Use of Non-Speculative Information Leakage 27
- III.3 Paper III - Doppelganger Loads: A Safe, Complexity-Effective Optimization for Secure Speculation Schemes 30
- III.4 Discussion 31

IV Unveiling Vulnerabilities: Breaking Existing Speculative Defenses 33
- IV.1 Paper IV - Data-Out Instruction-In (DOIN!): Leveraging Inclusive Caches to Attack Speculative Delay Schemes 33
- IV.2 Paper V - Reorder Buffer Contention: A Forward Speculative Interference Attack for Speculation Invariant Instructions 36

V Conclusion 38

Svensk Sammanfattning 40

VI References 45

Acknowledgments

As I sit down to write these acknowledgments, I am filled with a profound sense of gratitude for the individuals who have made this journey possible. Their support, encouragement, and unwavering belief in me have been the driving force behind the completion of this thesis.

Growing up in Nea Anchialos, a very small town in Greece, during the Greek economic crisis, initially failing to get admitted to Electrical and Computer Engineering (ECE), my teenage self could have never imagined what the future held for me. Graduating from ECE, getting admitted to a PhD program and moving to Sweden, attending conferences, traveling to places I would have never thought of, and eventually writing my PhD thesis, are some of the achievements my young self who focused on basketball and computer games would have been impossible to comprehend.

It's often said that a PhD journey is full of ups and downs. That is true. What comes with more ups and downs is life. I could write so many stories from my PhD journey, moments where life got dark and sad, situations where my anxiety and stress took over, and times where I was overjoyed beyond measure.

But as with every chapter in life, what made it memorable is not the struggles or the accomplishments, those are lessons and experiences. What made it memorable is the people that were around.

I would like to thank Stefanos, not only for offering me the opportunity to join Uppsala University and guiding me throughout my PhD journey, but mostly for being the person he is, working with him was a charm. I would also like to thank Magnus for our endless discussions and all the work he put in, and Amund for being a great close collaborator.

Particularly to all the UART members, past and present, thank you for the help and support. I would like to express my greatest thanks to Chris, for welcoming me to UART and in Uppsala, being my first PhD collaborator and co-author, but mostly for being a good friend since day one, thank you. To all my friends who made (and make) my life in Uppsala fun, thank you from the bottom of my heart. To all my friends outside of Sweden, I know that we may not see each other that much anymore, but I am glad for all the support and endless conversations we still hold.

Thank you to my parents Artemisia and Konstantinos, who have supported me unconditionally, my brother Fanis for constantly showing me the greatest love and support, and my girlfriend Linnea for all her love.

I. Introduction

In recent years, computer architecture security has taken considerable attention, particularly since the disclosure of the Spectre [19] and Meltdown [23] attacks in 2018. These attacks exploit speculative execution, a feature within central processing units (CPUs), to leak data that are normally inaccessible during program execution. To address this challenge, numerous defenses have been proposed, albeit at the expense of significant performance degradation and design complexity. This thesis approaches the issue from two different perspectives. Firstly, it endeavors to reclaim lost performance by implementing novel techniques on established state-of-the-art mechanisms. Secondly, it proves that some of the existing mechanisms are still vulnerable, by demonstrating novel attack scenarios on them.

Out-of-order execution refers to the ability of modern processors to execute instructions in a different sequence than their original program order, executing them as soon as they are ready, rather than waiting for all preceding instructions to complete first. With out-of-order execution younger instructions that are ready to execute can bypass older instructions that are not ready to execute. As a result, when encountering instructions that need to make a decision (e.g., conditional branch instructions), the condition state may be unknown if it depends on preceding instructions that have not yet executed and becomes challenging to determine the exact path of execution moving forward. Instead of waiting until all previous instructions have finished executing, processors rely on various heuristics and past execution patterns to make educated predictions and continue the program execution. This particular feature, where instructions are speculated and executed along paths that may involve uncertainty regarding the correctness, describes, in essence, the term *speculative execution*. Speculative execution is one of the most important features in modern computer architectures that improve the overall system performance dramatically, since it allows the processor to continue execution, without stalls, usually with high correctness on the predictions.

Certainly, there are cases where the processor makes incorrect predictions and executes instructions following the wrong path. Processors refrain from committing or retiring speculatively executed instructions; instead, these instructions remain in the pipeline until the processor determines the actual outcome of the predicted path. If the prediction is correct, the instructions are committed, and the execution continues as normal. On the other hand, if the processor executes along the wrong path, it discards any speculative alterations and recovers all architectural states as if speculative execution never occurred.

The architectural state refers to the externally visible state of the processor as perceived by the software or the programmer, such as the registers, which are directly accessible and manipulated by the software running on the processor. The microarchitectural state refers to the internal state of the processor that is not directly visible to software, such as the contents of the data cache. Upon a misprediction, even though the architectural state is recovered, the microarchitectural changes remain, leaving traces behind that can be exploited by potential attackers. The microarchitectural state is not accessible by the software layer and typically a malicious adversary needs to form a side-channel in order to leak such information. Such a side-channel could potentially be a timing side-channel (e.g., measuring the latency of a memory access). By forming a timing side-channel, a malicious adversary can extract hints about the program behavior, for example determine the memory access pattern of a victim program by monitoring which addresses the victim accesses, or even specific data values that could potentially be secrets, by de-referencing memory locations and creating observable timing differences based on those values.

Combining speculative execution and side channels, speculative side channels emerge. In a speculative side-channel attack, a secret is improperly accessed speculatively (during wrong path execution) and then leaked by passing it to a transmitter instruction while still under speculation. Even though misprediction is identified and the architectural state is recovered, the microarchitectural state remains and can be exploited. In a later phase, an attacker forms a side-channel (like the timing side-channel discussed above) and leaks the value of the secret, for example, by measuring the time of a specific cache line access.

As made clear from the preceding paragraphs, the crux of speculative side-channel attacks resides in speculative load instructions. During speculative execution, the processor has a "free pass in memory". A way to tackle speculative attacks would involve eliminating speculative execution completely, but that would draw back performance improvements over the last decades. Another naive solution would be to delay the execution of all speculative load instructions, but this also would incur extreme performance overheads [32]. Hence, more sophisticated defenses were introduced, facing the problem from completely different points of view. These defenses explore techniques such as buffering [38], restricted propagation [36], selective delaying [32], taint tracking [41], and numerous others. While these techniques succeed in mitigating performance overhead to varying degrees, they typically fail to eliminate it entirely, introduce considerable design and implementation complexity, and sometimes fall short of fully addressing security vulnerabilities. This thesis, and Papers I-V, tackle exactly all of those issues.

Papers I-III focus on recovering lost performance, by proposing simple and practical designs. Paper I introduces Janus [2], an approach that focuses explicitly on reducing implementation complexity while still recovering lost performance over previous proposed solutions. Janus uses a lightweight rein-

forcement learning agent to dynamically adjust to the program behavior and select the most optimal, performance-wise, defense to run as the underlying mechanism. Even though reinforcement learning is connected with implementation complexity, Janus uses the simplest and lightest form of reinforcement learning introducing negligible design overheads. Janus selects between two of the simplest defense ideas that can co-exist in a single processor design and shows that they can also be complementary. The paper also introduces an attack concept based on how cache hits and misses can influence the behavior of upcoming memory instructions. Paper II introduces ReCon [3], an optimization for existing speculative-side channel defenses that leverages non-speculative information leakage (leakage that occurs during the normal execution of the program) to improve the performance of the underlying defense mechanism that restricts speculation. ReCon tracks non-speculative information leakage, marks data in the memory hierarchy as revealed (public information) and concealed (potential secrets), and uses this information to enable more memory-level parallelism while still under speculative restrictions. Paper III introduces Doppelganger Loads [20], also an optimization for existing speculative-side channel defenses. Doppelganger Loads leverage address prediction to enable the execution of speculative loads independently of secrets, thereby minimizing the impact on performance caused by restricted speculative execution.

Papers IV and V focus on addressing security vulnerabilities in previously proposed side-channel defenses. Paper IV introduces DOIN! [4], an attack that leverages inclusive caches behavior to make data and instructions interfere in the lower level caches and create observable differences in the data cache without actually using data, but instructions — since using data is what specific defenses restrict you from. Hence, even if a defense does not allow the execution of load instructions to transmit information in the data cache, DOIN! shows that an attacker can fetch any type of instruction and still create observable timing differences in the data cache. The paper also proposes a simple solution to such an attack by including instructions in the protection scheme. Lastly, Paper V introduces Reorder Buffer Contention [5] an attack that fills the reorder buffer on demand based on a secret value and pushes in or out, controlling its execution, a transmitter instruction. Reorder Buffer Contention can create observable timing differences and leak information by accessing a completely independent, to the control and data flow path, instruction, if the underlying defense allows you to do so. Moreover, in Paper I an attack is introduced that leaks information by forming a channel based on secret-dependent cache hits and misses, without using any other explicit instruction (such as, for example, a control instruction).

This thesis presents five works in total. Three of them, namely Janus, ReCon, and Doppelganger Loads, focus on *enhancing defenses* in terms of performance and/or complexity, and two of them, namely DOIN!, and Reorder

Buffer Contention (and an attack within the Janus work), focus on *unveiling vulnerabilities*.

II. Background and Related Work

This thesis discusses security concepts regarding speculative side-channels. Even though there exist several side-channels that an adversary can form to leak information (e.g., energy usage), in this thesis we will focus mostly on cache side-channels that can be exposed from the software layer and are easier to exploit. An attacker leverages speculation to acquire secrets that are otherwise inaccessible and uses the cache side-channel to transmit the information.

Papers I-III propose optimizations that built on-top of existing speculative defenses, and Papers IV and V unveil vulnerabilities of existing speculative defenses. More specifically, Paper I combines two existing mechanism ideas, Paper II and Paper III propose transparent optimizations that can be applied on a variety of different defenses, and Papers IV and V unveil vulnerabilities in prior research proposals. This section presents the required background for speculative execution and cache-side channels, explaining the concept of speculative side-channel attacks (i.e., Spectre [19]), and presenting state-of-the-art research regarding speculative side-channels.

II.1 Speculative Execution

Speculative execution is a fundamental technique employed in modern processor architectures to enhance performance by executing instructions ahead of time, even in the presence of uncertainty regarding their necessity. It operates in conjunction with out-of-order execution, enabling processors to execute instructions in a non-sequential manner, bypassing older instructions that are not yet ready for execution. This approach aims to mitigate situations where the processor stalls without executing any instructions, and optimizes resource utilization. Take for example a conditional branch, such control instruction needs to compute a condition and determine if this condition corresponds to a *true* or *false* result to continue executing the correct code (the correct path of execution). Commonly, such branch conditions depend on memory accesses that may take thousands of cycles to be completed, leaving the branch condition unresolved at the same time and the processor unable to determine the next correct instruction. What speculative execution does is that it allows the processor to continue the execution without stalling on the branch by trying to predict the future. It essentially "guesses" what comes next.

During speculative execution, processors rely on various heuristics and past execution patterns to make educated predictions and continue program execution, despite potential branching complexities and uncertain outcomes.

However, speculative execution introduces security vulnerabilities. In occasions where a prediction is made and the processor is executing instructions along an uncertain path, these instructions are retained in the pipeline until the actual outcome of the speculated path is verified. If the prediction is correct, the instructions are committed, and execution proceeds as intended. Conversely, if the processor follows an incorrect path, it resets all architectural states, disregarding any speculative changes, continuing from the correct path of execution. While the externally visible architectural state, such as registers, is restored upon misprediction, internal microarchitectural alterations, such as data cache contents, persist.

These microarchitectural changes resulting from wrong path execution during speculative execution can compose traces that potential attackers may exploit. Since these microarchitectural states are not directly accessible to software layers, adversaries often rely on side-channel attacks, such as timing side-channels (which is also the focus of this thesis) to extract sensitive information. Through timing side-channels, adversaries can identify either the program behavior and/or specific data values.

II.2 Cache side-channels

Caches offer a significant route through which the memory hierarchy can be exploited to disclose information. The cache hierarchy has been a known side-channel for a long time, and still remains one of the easiest and most accurate microarchitecture features to exploit. Easiest because it can be accessed by software without the need to have physical access to a machine, and most accurate because different levels of caches come with different access latencies.

Prime+Probe [27] is a cache side-channel attack that allows for the observation of changes in particular cache sets. The attack requires the malicious attacker to load cache lines into a particular cache set (prime step), wait for a predefined period of time, and then measure the access time to these cache lines (probe step). By monitoring the timing alternations resulting from cache line evictions, the attacker can infer whether the victim accessed the targeted cache set during the waiting period. For example, since the attacker primes the set, cache hits are expected when this set is accessed anew on all the probe accesses. Cache hits translate to fast memory access when measuring the time in cycles. In case of an observed miss (long latency access when measuring the time in cycles), the attacker can infer that the victim accessed this cache set and caused an eviction in one of the primed addresses. This attack strategy can be utilized to extract encryption keys by observing the memory access pattern [9].

Expanding on this approach, LLC Prime+Probe [24] applies the Prime+Probe attack to the Last Level Cache (LLC). By capitalizing on the shared nature of the LLC across multiple cores, this technique enables the attacker to target a victim operating on a different core. Information leakage occurs when the victim evicts data that the attacker has primed within the LLC.

While Prime+Probe can provide indications of whether a specific set was accessed, it cannot provide detailed information about the victim's accesses, like values. Nonetheless, Prime+Probe remains a powerful attack method as it does not necessitate shared address space between the victim and the attacker, but requires only shared structures.

In contrast, when the victim and the attacker do share the same address space, attacks such as Flush+Reload [39] are feasible. In this scenario, the attacker targets a specific cache line, flushes its contents from the cache, allows the victim to run for a period of time, and then reloads the cache line, measuring its access time. Depending on whether there is a cache hit or miss, information regarding whether the victim accessed the specific cache line is leaked. Similar attacks with analogous concepts on shared memory include Flush+Flush [13], Evict+Reload [14], among others.

II.3 Speculative Side-Channel Attacks

Speculative side-channel attacks take advantage of the wrong path execution during speculation (Section II.1) to access memory that would otherwise be inaccessible, forcing the processor to leave microarchitectural traces during this wrong path execution, and then use a side-channel to leak this information (for example the cache side-channel as explained in Section II.2, applying the Flush+Reload [39] attack). Spectre attacks [19] is exactly this: leverage wrong path execution to leak information that would otherwise be inaccessible.

II.3.1 Spectre Attacks

The process of a typical Spectre Variant 1 (V1) is illustrated in Figure II.1. Spectre V1 leverages a conditional branch instruction to force the program into speculation.

For the setup, the attacker trains the branch predictor (line 13) so that it always predicts the condition in the if-statement in the access_array() function as *"true"* (line 2) so that it follows the *taken* path of execution and loads a value from the memory (line 3). The attacker also flushes an array from the cache (line 14) that will be used to determine the value of the loaded secret. Because the branch predictor is trained as such to predict the taken path even if the condition is false, the attacker can perform an out-of-bounds illegal access (line 3 by the call of line 15), fetching a value from an address that is typically off-limits, accessing data that would otherwise be inaccessible. Even tho this

```
1  void access_array(int index){
2   if(index < array_size)
3    secret = array[index];
4  }
5
6  void train(){
7   for(i=0; i<100; i++)
8    access_array(0);
9  }
10
11 void attack() {
12  char probe_array[N * CACHE_LINE_SIZE];
13  train();
14  flush(&probe_array);
15  secret = access_array(secret_location);
16  x = probe_array[secret * CACHE_LINE_SIZE];
17  probe(&probe_array);
18 }
```

Figure II.1. Spectre V1.

false prediction will later be identified, and the processor will recover the architectural state by squashing all the instructions that were fetched from the wrong-path of execution, the attacker has enough time to acquire and leak the secret information (create observable traces in the microarchitectural state), before the speculation window closes.

To transmit the value of the secret, the attacker uses it as an address input into the probe array (line 16), thereby accessing a specific cache line. Depending on the secret's value, a particular cache line will be inserted into the L1 cache. By measuring the access latency of each cache line in the probe array, even after speculative execution has finished, it becomes possible to discern which cache line has been brought into the L1 cache, and consequently, the value of the secret (since it is used to create the address itself).

Following the verification of misspeculation, timing all cache lines of the probe array reveals that the cache line corresponding to the secret value exhibits a hit (lower access time), while the others miss (line 17). This discrepancy arises because the cache line accessed using the secret value remains in the L1 cache, unlike the other lines, which were flushed prior to the attack and are still absent from the it.

Besides Spectre V1, there are more Spectre variants. For example, Spectre V2 injects the branch target buffer (BTB) to continue the execution from a specific path executing instructions and accessing information that would otherwise be inaccessible, and Spectre V4 [15], exploits the memory disambiguation unit that allows the speculative execution of load instructions by guessing if they take the value from the memory hierarchy or forwarded by a preceding store.

Other Variants:
Spectre attacks have inspired researchers to further exploit speculative execution and propose novel attacks against different hardware modules. PACMAN [28] breaks ARM Pointer Authentication by trying to authenticate pointers speculatively, avoiding crashes that occur upon failed authentication. SMoTherSpectre [10] exploits speculative execution through port contention, I See Dead μops [29], Leaky Fronteds [12], and UC-Check [18] leak information through micro-op caches, and Phantom [37] exploits speculation that happens before the decode stage from the majority of the instructions.

Meltdown:
Meltdown [23] is another category of speculative side-channel attacks, which encompasses attacks that bypass hardware barriers, such as kernel permissions. The original Meltdown [23] attack exploits the absence of page permission checks before memory access. Hence, a malicious attacker can bypass the permissions because of vendor implementation details, and speculatively access memory that would otherwise be inaccessible. Meltdown attacks fall into a different category than Spectre attacks and are not considered in this thesis.

II.4 Speculative Defenses

Several speculative side-channel mitigations have been proposed as defenses [38, 31, 41, 36, 32, 17, 22, 11, 7, 6, 25] against attacks like Spectre [19]. Defense mechanisms can vary significantly depending on how they mitigate the problem, the complexity, and the performance overhead they introduce.

II.4.1 Invisible Speculation

One idea, that was common within the first wave of defenses, is to utilize buffers to temporally save the speculative results, before promoting them and exposing them to the actual system [38, 7, 31, 17]. In those defenses, speculative execution can continue as normal without changing the cache hierarchy. When speculation is verified, loads that have brought information into the temporary buffers are replayed by a new request [38, 31], so that the data are also cached.

II.4.2 Speculative Delay Schemes

An alternative security measure involves selectively delaying instructions that leak sensitive information during speculative execution. This method aims to prevent the transmission of secret data by delaying specific instructions

that could expose it to the non-speculative domain. The effectiveness of this technique against various threats depends on the type of instruction being delayed, such as resource contention (e.g., floating-point arithmetic instructions) or cache side-channel attacks (e.g., load instructions).

Non-speculative Data Access (NDA)
A naive proposal of this idea is proposed in Non-speculative Data Access (NDA) [36], where instructions that may speculatively access and bring secret information inside the core (e.g., load instructions), are not allowed to propagate their value to subsequent instructions while still being speculative. As a consequence, all transmitter instructions will never have their operands ready to execute while under speculation, and thus stall. Yet, with such a solution these values are also not propagated to non-transmitter instructions, restricting the overall execution and affecting significantly instruction-level parallelism (ILP).

NDA includes more strict proposals to cover a wider range of threats, for example not propagating the outcome of any speculative instruction. *In this thesis, we specifically focus on the NDA proposal referred to as Permissive Propagation, which marks only load instructions after an unresolved branch or store as unsafe.* The key intuition for this policy is that only loads can introduce new secret values into the microarchitecture [36].

Speculative Taint Tracking (STT)
Speculative Taint Tracking (STT) [41] shares the same idea with NDA, in the sense that it tries to delay the transmitter instructions and avoid exposing the secret. STT employs a taint tracking mechanism similar to dynamic information flow tracking (DIFT) [34] to prevent the execution of instructions that might disclose secrets (the transmitters). It taints the destination register of a speculatively load instruction, and keeps propagating this information to any subsequent dependent instruction. STT automatically untaints the destination register and all tainted registers that originate from this register, as soon as the corresponding load instruction becomes safe, i.e., when it becomes non-speculative. With a taint tracking system the secret value keeps propagating to subsequent instructions along with the taint mark, and thus any instruction that has tainted input and can leak information should be considered a *transmitter instruction* and should be specially handled.

Load transmitter instructions are handled by delaying their execution while speculative. STT refers to those as explicit channels [41]. A more implicit way of leaking information is to use the secret loaded data as input to a set of more complex instructions and infer its value by observing the program's behavior. For example, an attacker could use a secret-dependent branch (e.g., `"if(secret){body;} else{body2;}"`) that consists of two distinct paths. By observing the behavior of the program, the secret can be implicitly referred as the two different paths will cause different microarchitectural

changes. Those channels are called implicit channels. STT conducts a thorough examination of such channels, called explicit (such as those exploited by certain Spectre variants [19]) and implicit (such as SmotherSpectre [10]) channels, that could potentially leak secrets . Against explicit channels, STT delays the execution of any instruction transmitting data if its operands are tainted, ensuring tainted inputs do not influence execution until they are untainted. To mitigate implicit channels, STT introduces novel ideas for resolution-based and prediction-based mitigation mechanisms. For example, when it comes to conditional branches, it ensures that the prediction is secret-independent, as well as that the branch resolves when speculation is verified, when the secret is not a secret anymore.

Delay-on-Miss (DoM)

One more defense mechanism that focuses on selective delaying is Delay-on-Miss (DoM) [32]. DoM specifies that load transmitters leak information through the cache hierarchy only when the memory access misses. The reason is that a miss eventually turns into a hit and that creates observable timing differences (recall Flush+Reload [39] attack from Section II.2). DoM delays a speculative load instruction only when it misses in the first cache level. In case of a cache hit, DoM delays any side-effect that can leak information, allowing the value to propagate to subsequent dependent instructions as safe during speculation. Eventually, speculation is verified and the delayed load accesses are allowed to continue their execution. By delaying speculative load transmitters, DoM does not allow the creation of observable timing differences.

Since DoM misses the concept of speculative dependent instructions, as in STT, and operates by only allowing/permitting speculative load cache accesses, it is able to not only protect against leakage caused by speculatively accessed data (such as in STT and NDA), but also leakage from data (potential secrets) that have been loaded to registers prior to speculation (non-speculative accessed secrets). A secret that has been loaded into a register before speculation has started, in order to be leaked speculatively, it only requires a single load instruction to use the register as part of its address and turn a cache miss into a cache hit. DoM permits this behavior, in contrast to NDA/STT.

The following listings specify the difference in threat models of NDA/STT and DoM regarding register protection:

```
1 // NDA/STT
2 id = key[13];
3 if(cond){
4     acc = arr[id]; // Miss -
          Executes
5
6     tr = probe[acc]; //
          Delayed
7 }
```

```
1 // DoM
2 id = key[13];
3 if(cond){
4     acc = array[id]; //
          Miss - Delayed
5
6     tr = probe[acc];
7 }
```

In NDA and STT only speculative accesses of loads that have their operands ready (NDA) or untaint input (STT) can be performed. Since the variable `id` is loaded pre-speculation and both NDA and STT guarantee protection from speculatively accessed secrets, both can use it as input and access `array[id]` without restrictions – leaking the value `id` that has been loaded pre-speculation. On the other hand, DoM is more generic and delays all loads that miss in the cache (even if they are not transmitters). Hence, this behavior is able to protect against registers loaded pre-speculation since the access to `array[id]` will not be able to create observable timing differences if it misses – leaking the value of `id`.

Speculative Privacy Tracking (SPT)

Speculative Privacy Tracking (SPT) [11] offers comprehensive protection against speculative side-channels. While SPT and STT share the foundation of dynamic information flow tracking (DIFT) [34], SPT is different in the sense that it safeguards against all forms of leakage during speculation, including non-speculative secrets such as register values that have been loaded pre-speculation and have not been leaked.

In contrast to STT's focus on speculative tracking through registers, SPT introduces a global, continuous taint tracking mechanism that not only tracks both non-speculative and speculative accesses but also extends the tracking in the memory hierarchy. This innovative approach efficiently propagates taint/untaint information, ensuring protection against speculative leakage. SPT's taint tracking mechanism operates dynamically, distinguishing leaked data during both speculative and non-speculative execution phases. Upon identifying leaked data, SPT's untainting mechanism dynamically removes taints from dependent instructions, allowing them to execute normally under speculation. *SPT adheres to the principle that any data susceptible to leakage during non-speculative execution should not be treated as secret during speculative execution.* By maintaining taints in both non-speculative and speculative execution and dynamically untainting leaked data, SPT effectively mitigates the risk of leakage from all non-speculative secrets, including addresses that have not previously leaked their values non-speculatively. For example, using the similar code snippet as before:

```
1 // SPT
2 id = key[13];
3 if(cond){
4     acc = array[id]; // Delayed
5     tr = probe[acc];
6 }
```

SPT uses a global, continuous taint tracking mechanism, that captures both non-speculative and speculative execution (red variables) and is able to determinate as transmitters both access `array[id]` and `probe[acc]`.

Other Defenses
Furthermore, DOLMA [25] combines STT and DoM for a wider threat model. Since STT propagates taint information, it can basically delay any transmitter instructions (even though in this thesis we focus on cache side-channels), but still lacks register protection. DOLMA essentially applies DoM to enhance STT with register protection.

Lastly, proposals to undo the wrong-path speculation have been proposed [30]. Those defenses, track all speculative changes and recover the state if wrong-path execution has been detected, essentially leaving no observable trace.

A Discussion About Threat Models
All security defenses must compromise trade-offs, balancing their threat model, security guarantees, and performance. These compromises are crucial in determining a defense mechanism's suitability for a given scenario.

For instance, consider the distinction between STT and DoM. STT offers the potential for broader side-channel coverage by identifying and delaying a wider range of instructions marked as transmitters. In contrast, DoM specifically targets attacks that exploit the memory hierarchy.

A key limitation of STT is its inability to protect against information leakage that occurs when secret data is loaded into registers before the speculation window begins. DoM, however, can provide protection against such threats.

Similarly to STT, a particular version of NDA, designed to prevent values from propagating to subsequent dependent instructions, can effectively mitigate information leakage across all side-channels. This is because secrets cannot be propagated to any transmitter instructions. Nevertheless, much like STT, this NDA variant does not safeguard against secrets loaded into registers pre-speculation.

SPT builds upon the STT threat model by extending information flow tracking to encompass all execution, both speculative and non-speculative. This allows SPT to protect registers loaded prior to speculation, albeit with a significant impact on performance. Consequently, SPT revises STT's security guarantees from protecting all speculatively accessed data to protecting all speculatively accessed data except for information that has already leaked non-speculatively.

In this work, we focus on protecting against cache side-channels. This is the same model as the threat model assumed in DoM. For fairness, we consider an STT implementation that considers transmitters only load instructions (similar to prior work [40]), so that they share the same threat model.

II.4.3 Enhancing Speculative Defenses

Speculative Data-Oblivious (SDO)
Speculative Data-Oblivious (SDO) [40] optimizes STT by enabling data-oblivious execution. SDO leverages cache-level prediction, which is trained in a non-

speculative manner. SDO executes load instructions, known as oblivious loads, to speculatively retrieve a value from a predicted location within the cache hierarchy. Essentially, SDO predicts the cache level the value is expected to hit in a secret-independent manner.

Speculation Invariance (InvarSpec)

InvarSpec [43] is a hardware/software co-design that identifies load instructions that will commit regardless of the speculation's outcome, allowing them to lift their protection while still speculative. Those instructions are called speculation invariant instructions.

Such speculation invariant load instruction can be seen in the following listing (line 8):

```
1  if(cond){
2  // Speculation window starts
3  // -------------------
4     var = t;
5  }
6  data_dependent = array[var];
7  id = 10;
8  speculation_invariant = sort[id]; // Speculation invariant load
9  // -------------------
10 // Speculation window finishes
```

Assume that the branch `"if(cond)"` starts a long latency speculation window, all subsequent instructions entering the pipeline are speculative and can eventually be squashed, thus speculative defenses are applied. In reality, though, only instructions `"var = t;"` (line 4) and `"data_dependent = array[var];"` (line 6) are dependent to the speculation, the former having a control dependence to the branch, and the latter having a data dependence on a control-dependent instruction. Those two instructions are the ones that can be used to follow the wrong path execution and leak information. Instructions `"id=10"` (line 7) and `"speculation_invariant = sort[id];"` (line 8) have no actual dependencies to the speculation part itself. Their execution is going to be the same regardless of the outcome of speculation, which means even if they are squashed they will be issued and executed exactly the same as the processor continues its execution after the squash. InvarSpec tracks these instructions, that are independent of the speculation outcome, and enables their execution while still speculative.

To accomplish this, InvarSpec keeps track of all control and data dependencies, identifying which instructions are independent of the speculation part. Then, it uses this information on-top of existing secure speculation schemes like InvisiSpec [38] and DoM [32] to enable the execution of instructions while still speculative. Notably, this addresses a significant performance bottleneck for DoM, which delays all memory accesses that miss in the memory hierarchy. With InvarSpec, several of those misses can still be executed while under speculation.

Other Optimizations

Moreover, there are some optimizations that focus on reducing the work done by the underlying defense schemes. That can be achieved by managing to have shorter speculation windows, so the defense is enabled for a much shorter time and less dynamic instructions are affected by the mitigation strategy.

Clearing the Shadows [35] focuses on reordering instructions to minimize speculation windows. For example, if the speculation window length caused by a branch instruction depends heavily on a previously long latency load, it will move the load instruction in an earlier stage so that it is finishes earlier, enabling the branch to resolve faster and consequently lead to a much shorter speculation window. Clearing the Shadows leverages a compiler and hardware/software co-design. In the same context of reducing the speculation window, but targeting a specific cause of speculation, Pinned Loads [42] targets the associated overhead of memory reordering, proposing a mechanism to resolve memory violations promptly for earlier execution of restricted loads. Both Clearing the Shadows and Pinned Loads focus on eliminating speculation to enhance performance by reducing the amount of work caused by the underlying mitigation, and not optimize the defense mechanisms themselves.

II.5 Attacks on Speculative Defenses

Even though defenses aim to provide security guarantees, they usually fail to accomplish it introducing new vulnerabilities and allowing for further exploitation. This is an expected behavior [26] as to completely remove the speculative threat, defenses need to completely remove the effects of speculative execution. Proposed defenses that aim to reduce the amount of mitigation overhead, focus on relaxing those strict conditions offered by naive solutions making them easier to exploit.

Speculative interference attacks [8] present a new variant of speculative side-channel attacks. Unlike conventional speculative attacks, in speculative interference attacks, the side-channel instructions are present before speculation has even started, in the non-speculative realm. In these attacks, the attacker utilizes instructions from within the speculation window to interfere with older non-speculative instructions placed before the speculation window. This interference causes different timing variations in terms of when instructions execute in time. Using a secret value to cause this interference, it is possible to implicitly leak the value of the secret by observing the program behavior, and as a result, information is leaked from the speculative domain to the non-speculative domain. After their disclosure several works were proposed to defend against them, mainly enforcing proper execution priorities [6, 33, 25].

Additionally, delaying execution through taint tracking [41] was shown to be vulnerable to information leakage via secret-dependent store instructions exploiting the TLB, since store instructions were not initially considered trans-

mitter instructions [25]. This could be considered more as an implementation detail, as defenses such as STT are flexible in terms of which instructions to consider as transmitters. The simple solution is to include store instructions as a transmitter instruction in the initial STT design implementation.

Furthermore, undo-based speculation schemes [30] were found to be vulnerable to unXpec attacks [21]. Undo-based speculation schemes, essentially track the speculative changes in the memory hierarchy and recover the state after misspeculation is identified. unXpec shows that the size of the speculation window fluctuates based on the number of speculative loads that have to be recovered after misspeculation is identified. Hence, unXpec leverages secret-dependent paths that fetch a significant amount of load instructions and when it comes to undoing (recovering) speculation they take much more time than paths that have no speculative loads, leaking the executed path and consequently the value of the secret.

III. Novel Speculative Defense Tactics for Enhanced Performance

III.1 Paper I - Janus: A Simple and Efficient Speculative Defense using Reinforcement Learning

Paper I describes an architecture developed to mitigate speculative side-channel attacks effectively with improved performance.

The insight

JANUS highlights the complementary nature of Non-speculative Data Access (NDA) and Delay-on-Miss (DoM), noting that they offer distinct advantages in certain scenarios. NDA under the Permissive Propagation (PR) scheme never propagates the result of speculatively accessed loads. NDA explores Memory-Level Parallelism (MLP), since it can execute speculative load instructions in parallel, while it lacks Instruction-Level Parallelism (ILP) as the result of speculatively loaded values is never propagated to subsequent instructions. On the other hand, DoM exhibits no MLP, since all memory requests that miss in the first level cache are dropped, but exhibits partial ILP when memory accesses hit in the first level cache and the result is propagated to subsequent instructions. As characterized in the SPEC2017 benchmark suite illustrated in Figure III.1, *cactuBSSN, exchange2, leela, and x264* perform great when using DoM as the mitigation mechanism, whereas those benchmarks encounter notable challenges under NDA mitigation mechanism. On the other hand, *gcc, mcf, omnetpp, and xalancbmk* demonstrate improved performance with NDA compared to the DoM mitigation. The geometric mean shows DoM as more efficient defense with a slim 2.3% lead, but this average masks significant differences in individual benchmark performance, creating a misleading impression of comparable defense effectiveness.

Overview of JANUS

Paper I takes advantage of this potential complementary behavior of NDA and DoM by incorporating both mechanisms into one processor architecture, and by enabling a lightweight reinforcement learning (RL) agent, to switch between those two, it achieves an effective defense strategy. Essentially, it operates by employing two fundamental defense mechanisms: NDA and DoM, in conjunction with a lightweight reinforcement learning (RL) agent. This combination enables JANUS to dynamically adapt its defense strategy based on

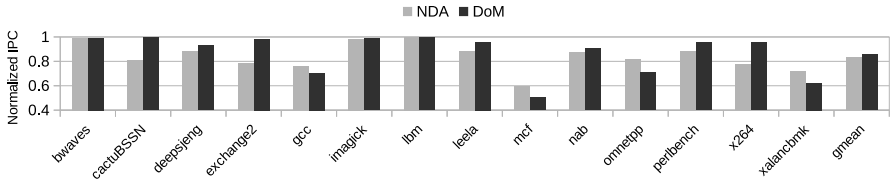


Figure III.1. Non-speculative Data Access (NDA) and Delay-on-Miss (DoM) performance normalized to unsafe baseline.

performance metrics within a specific time period. Note, that the two defenses never operate simultaneously, it is always one defense that operates within a specific time period and it is the agent's responsibility to choose which one.

The architecture is characterized by its simplicity, as it relies on defense mechanisms that are easy to implement. However, Paper I also suggests that Janus has the ability to incorporate more complex defense mechanisms, considering resource constraints. A key feature of Paper I is its ability to continuously learn and refine its defense strategy through the RL agent. This adaptive learning mechanism allows JANUS to select the most optimal defense mechanism based on real-time performance feedback, thereby maximizing its effectiveness in defending against speculative attacks. Despite its simplicity, JANUS demonstrates remarkable adaptability.

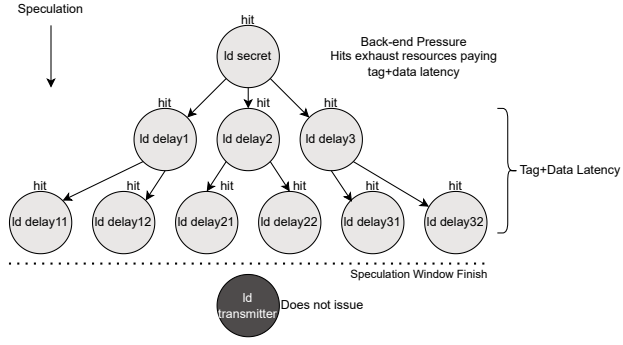
In terms of performance, JANUS surpasses traditional baseline defenses like NDA and DoM, as well as more complex mechanisms such as STT and state-of-the-art optimizations. On average it outperforms other proposed schemes by 2.2 percentage points on SPEC2017 benchmark suite, and 4.7 percentage points on SPEC2006. By minimizing performance degradation while providing a lightweight design, Paper I proves to be a highly efficient solution for mitigating speculative side-channel attacks.

Despite its effectiveness, JANUS maintains a low storage overhead of only 16 bytes and a complexity-effective design, making it suitable for practical implementation in various computing environments.

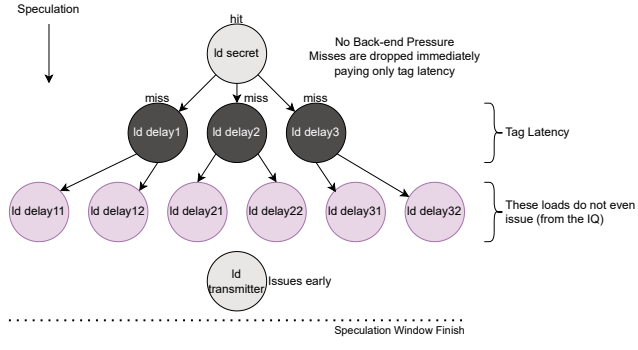
Moreover, the paper highlights the potential vulnerabilities introduced when combining different mechanism ideas during the same speculation window. It shows that such combinations can expose the architecture to potential attacks, and demonstrates a simple attack scenario on prior proposals [43, 16].

The attack utilizes two distinct features: being able to use the secret (e.g., as in DoM when the secret hits in the first level cache), and being able to create observable timing differences (e.g., as in NDA/STT where they always execute an independent speculative load instruction). To perform the attack, a channel based on cache hits and misses is formed. The attacker applies two steps: (1) use the secret to create secret-dependent addresses in chains of loads (think of it as a rooted tree that every node is a load instruction and the root is the node that loads the secret value – see Figure III.2), and (2) execute a

younger (in program order) independent load instruction that is placed after these chains of loads (the transmitter).



(a) The secret forms the address of the next level of dependent loads. A cascade of loads, that all hit (light shaded nodes), is triggered. The speculation is squashed before the transmitter has a chance to execute (dark shaded node).



(b) The secret forms another set of addresses but this time triggers all misses in the first level (dark shaded nodes), where the request is dropped. All further dependent loads (pink nodes) have to wait until their input operand becomes ready. The independent transmitter load (light shaded node) has a chance to execute early before the speculation is squashed, creating observable timing differences in cache.

Figure III.2. Attack scenario based on hit/miss implicit channel.

Since the attacker can load and use the secret value, and all the accesses on the chains of loads are initiated by this specific value, it is crystal clear that they all depend initially on the secret value. For the attack to succeed assume that all subsequent dependent loads (all the tree nodes) initiate all hits or all misses in the cache (this can also be set up by the attacker before applying the attack by caching or flushing cachelines). Essentially, a tree of loads that depends on a secret value may cause only hits or misses in the cache. This behavior can leak the value of the secret as it influences the time the subsequent (younger in program order) transmitter instruction executes. The attack is illustrated in Figure III.2 and further explained in the next paragraphs.

In step (1) depending on the secret value the chains of loads may all hit in the cache and keep the backend resources hot by executing all load instructions (values keep propagating down the chains and loads keep executing — traversing all the tree from root to the leaves), as illustrated in Figure III.2a, or may all miss in the L1 cache and hence requests are dropped (recall DoM – if L1 miss, drop their request — the tree is not traversed) and the chains of loads do not execute, since the source operands of upcoming load instructions are not ready and contention is not added in the backend resources, as illustrated in Figure III.2b. To sum up step (1), depending on the secret value several loads may execute, or no loads execute at all. In the former case, there is contention in the backend resources, in the latter case there is no contention in the backend resources.

In step (2), an independent load instruction (the transmitter) that comes after the chains of loads in program order, depending on the behavior of step (1), may or may not execute during the speculation window. If the chains of loads hit in the cache and keep executing, keeping the resources active, they leave no time for the transmitter to execute. On the other hand, if the requests are dropped because of an L1 cache miss and the loads do not have their source operands ready, the transmitter instruction is the only instruction ready to execute.

By accessing and measuring the latency of the load transmitter instruction after the speculation window has finished (a typical Flush+Reload [39] attack on the transmitter), we can infer if it was cached or not, which leaks the information of whether the chains of loads hit or miss in the cache, and consequently the initial value of the secret that triggered the addresses that hit or miss.

This attack can be applied to any defense mechanism that allows the use of the secret and the creation of observable timing differences. For example, InvarSpec [43] uses the value of the secret (since it builds on-top of DoM defense mechanism) and creates observable timing differences by executing the speculation invariant instructions.

III.2 Paper II - ReCon: Efficient Detection, Management, and Use of Non-Speculative Information Leakage

Paper II is an optimization for speculative side-channel defenses that removes speculative restrictions for non-secret addresses.

The insight

Speculative Privacy Tracking (SPT) [11] suggests that data leaking through non-speculative execution should not be treated as secret during speculative

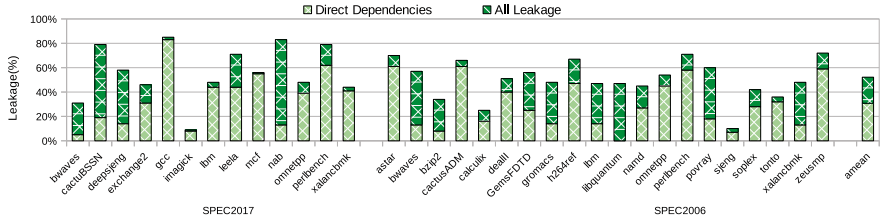


Figure III.3. Percentage breakdown of leakage out of all address space.

execution. Paper II builds on this definition and identifies a significant source of information leakage: load pairs that dereference pointers or index base-addresses. While various forms of non-speculative leakage can occur in program execution, Paper II focuses exclusively on exploiting the non-speculative leakage due to load pairs, offering a simplified yet effective solution.

Overview of RECON

Paper II's simplifies the complex task of detecting and managing non-speculative leakage into two straightforward functions: observing load pairs dereferencing pointers and marking the values of these pointers as either revealed or concealed in the memory hierarchy. We call this RECON (Reveal/Conceal). If two loads execute during the normal execution of the program (non-speculatively), with the second load dereferencing the value of the first load, RECON reveals the value of the first load as public – as dereferencing the first load means that its value is now exposed in the memory hierarchy. It is important to understand that this non-speculative leakage eliminates the need for speculative execution defenses, such as NDA and STT, which primarily exist to protect speculatively accessed data that could potentially be secrets. In cases where the value has not been revealed or is changed, it remains concealed as a secret. RECON achieves this by marking the address that contains the revealed or concealed value accordingly.

Moreover, RECON efficiently propagates conceal/reveal information through the cache hierarchy using the existing coherence protocol. Cache lines are tagged with reveal/conceal information, which is piggybacked on the coherence transactions of a standard directory MESI protocol. This enables effective management of leakage information throughout a multicore cache hierarchy without the need for protocol modifications.

Conventional approaches, based on general Dynamic Information Flow Tracking (DIFT) [34], aim to track every piece of data and its potential impact on information flow throughout the execution pipeline. While effective for comprehensive security, these mechanisms introduce significant complexity and overhead, RECON hones in on the most critical sources of leakage: non-speculative direct-dependent load pairs.

The rationale behind this approach is that a substantial proportion of information leakage indeed occurs through load-load pairs. The results illustrated in Figure III.3 indicate that, on average, 53% of the address space in SPEC benchmark suites leaks its content when we measure leakage by capturing it using Dynamic Information Flow Tracking (DIFT). Moreover, direct load-to-load dereferencing, which is ReCon's primary focus, is responsible for 32% of the address space that has leaked its content. This means that direct load dependencies alone cover 60% of the total leakage. In some cases, there is negligible additional leakage when measured with DIFT, and the program's leakage is solely due to direct-dependence load-to-load dereferencing. For instance, benchmarks like *gcc, imagick, mcf, and xalancbmk* from SPEC2017 fall into this category.

ReCon identifies non-speculative load pairs using a dedicated Load Pair Table (LPT). Each LPT entry contains an active bit and an address field, accessed using source and destination physical registers. When a load instruction commits, ReCon updates the LPT with the address of the loaded value. ReCon also checks the active bit that corresponds to the index given by the source register for potential load pairs. This mechanism ensures that the second load in a pair will match the entry that contains the address accessed by the first load of the pair.

ReCon tracks revealed and concealed addresses by associating a reveal/-conceal bit vector with each cacheline. Each bit corresponds to an 8-byte word of the cacheline. Newly fetched cachelines are marked as fully concealed. When a second load in a pair commits, it sends a reveal request to the L1 cache, setting the bit for the corresponding word in the cacheline, revealing the address.

To utilize the shared cachelines in multicore processors, ReCon integrates its bit-vector approach with the cache coherence protocol. This ensures that non-speculative leakage information can be shared among cores, enabling optimization opportunities.

ReCon optimizes speculative load instructions, enabling them to execute without unnecessary delays. If a speculative load accesses a revealed address, then it is guaranteed to access a non-secret, and special handling can be applied. For NDA, ReCon propagates such loaded value to dependent instructions. In the case of STT, ReCon untaints the destination register, enabling the use of the value by dependent instructions. These optimizations lead to increased instruction and memory-level parallelism, boosting overall performance.

We observe that one average ReCon reduces the performance overhead of NDA and STT by 45%.

III.3 Paper III - Doppelganger Loads: A Safe, Complexity-Effective Optimization for Secure Speculation Schemes

Paper III is a performance optimization for speculative side-channel defenses that focuses on recovering lost Memory-Level Parallelism (MLP) that occurs due to the restrictions in memory accesses while under speculation.

The insight

While non-speculative and speculative execution are two distinct processor features, they are highly correlated. It turns out that the majority of the loads start their execution while under speculation since there are various ways for a processor to enter speculative execution mode. In fact, more than 85% of loads that execute, execute while being speculative [32]. Obviously, in the average case the majority of those instructions turn out to be correct and eventually commit and retire from the processor. This means that restricted speculative execution turns to normal non-speculative execution later on. Paper III has the insight that this repetitive non-speculative behavior, which starts of as restricted speculative execution under speculative defenses and commits in a later stage, can efficiently unlock Memory Level-Parallelism (MLP) if it is remembered and used accordingly.

Overview of DOPPELGANGER LOADS

DOPPELGANGER LOADS, similarly to prior work [40, 32], claims that non-speculative accesses to the memory hierarchy are observable during program execution and this information can be used to enable speculatively restricted memory accesses, ensuring that it always happens in a secret-independent way. Leveraging this understanding, DOPPELGANGER LOADS employs address prediction to capture non-speculative execution patterns and uses this information to execute speculative loads that would otherwise be restricted under a secure speculation scheme. This strategy significantly enhances the performance of secure speculation schemes.

DOPPELGANGER LOADS ensures that predictions are based on non-speculative data by training the predictor only on committed instructions. This approach protects against leaking secrets, as the predictor cannot issue a prediction on information that has not been trained on, which means information that is illegal and accessed only during speculative execution (secrets) cannot affect the state of the predictor. When a speculative load enters the pipeline, DOPPELGANGER LOADS predicts its address and issues a doppelganger load if confident in the prediction. Once the actual address is generated according to the underlying secure speculation scheme, it is checked with the predicted address. If they match, the actual load completes upon the doppelganger load's execution, which began several cycles earlier. If not, the actual load is issued immediately, as this exact moment is where the speculative defend mechanism

would issue this particular load either way. DOPPELGANGER LOADS does not affect the timing the underlying mechanism makes decisions. Essentially, DOPPELGANGER LOADS tries to predict the address earlier and thus start the execution of a load before the underlying scheme allows it to, in a secret-independent manner. As a consequence, when it is ready to execute from the underlying scheme (may be several cycles later than an unsafe baseline due to restrictions), DOPPELGANGER LOADS may have already finished its execution. DOPPELGANGER LOADS prepares and executes load instructions before it is needed to do so, but never propagates or makes other decisions before the underlying mechanism allows it to.

DOPPELGANGER LOADS introduces minor modifications to existing load store units (LSU), primarily adding a predicted bit to load queue entries to indicate predicted addresses and initialize comparisons when actual addresses are generated. It utilizes existing registers and load address fields, as doppelganger and actual loads never coexist. Existing prediction structures, such as next-line or stride-based predictors used in caches for L1 prefetching, are repurposed to minimize the overhead of address prediction. Modifications include delaying training until data is non-speculative and tracking and predicting multiple speculative load instances. Consequently, DOPPELGANGER LOADS incurs only minimal design overhead compared to conventional address prediction methods.

Results show that DOPPELGANGER LOADS can reduce the overhead introduced by DoM, NDA and STT by 40% on average.

III.4 Discussion

The first part of the thesis focuses on performance enhancements by utilizing existing mitigation techniques. When combining mitigation techniques it is worth highlighting the trade-offs between security and performance.

STT protects against all speculative accessed data, but STT+ReCon protects against all speculative access data **minus** those that have leaked non-speculative (non-secrets), for a significant 45% overhead reduction.

Janus focuses on protecting leakage occurring from the memory side-channel, but still incorporates the NDA idea that is suitable for protecting against more side-channels than just the memory side-channel. Recall that in this thesis we focus on cache-side channels — transmitter instructions are only load instructions. NDA does not propagate a potential secret value to any subsequent instruction, having a much broader range of protection compared to this thesis's threat model. For example, NDA can also protect against contention attacks that use the floating-point unit to leak information. When it comes to cache side-channels, though, NDA is not the best-performing solution as its wider threat model integrates more limitations and introduces higher performance overheads. Janus has the insight that NDA can still perform better than

side-channel-specific defenses (e.g., DoM) on specific types of workloads, as in those that face substantial amounts of speculative load misses. Since Janus utilizes both defenses, it downgrades its threat model to the minimum guarantees and is limited to protecting only against cache side-channels (i.e., DoM).

In the same work, Janus also takes advantage of DoM which guarantees protection against secrets that are loaded in registers before speculation has begun. Since Janus, combines NDA (no register protection guarantees) and DoM (register protection guarantees), it falls back to the minimum provided protection which is the one offered by the NDA mechanism (no register protection guarantees). Hence, Janus cannot guarantee register protection, even though in certain speculation windows it defaults to DoM that does. This is yet another example of trading security guarantees for performance gains.

It is widely accepted that not all security concerns have the same importance, yet a methodical quantifiable method of measuring the trade-off between security and performance is still missing from the literature and would create an interesting field of research.

IV. Unveiling Vulnerabilities: Breaking Existing Speculative Defenses

IV.1 Paper IV - Data-Out Instruction-In (DOIN!): Leveraging Inclusive Caches to Attack Speculative Delay Schemes

Paper IV describes an attack that is able to break defenses like Delay-on-Miss.

The insight
Defenses such as Delay-on-Miss protect against speculative attacks by delaying speculative data accesses that create observable timing differences in the first level data cache. A pair of inclusive caches means that lower level caches contain the information of higher level caches. For example, an inclusive pair of L1 and LLC means that LLC cache also contains the contents of L1 cache. Lower level caches (e.g., LLCs) share both instructions and data. This combination of the first level caches (both data and instruction) being mapped to the lower level LLC cache (inclusivity) and co-existing (both instructions and data co-exist in lower level caches) can be exploited to leak secret information by causing speculative data cache misses (that Delay-on-Miss protects against) when evicting data from the data cache by speculative instruction fetching that conflict in lower level caches.

Overview of DOIN!
Paper IV discusses an attack that utilizes inclusive caches to attack processor designs by incorporating conflict behavior between both data and instructions in lower-level caches (e.g., Last-Level Cache), that hold both data and instructions, to create observable timing differences in first-level caches (e.g., data cache). The way data and instructions conflict in caches during the attack is shown in Figure IV.1. Such an attack is also capable to break speculative defense schemes that aim to protect against leaks in the data cache.

This attack, called DOIN! (Data-Out Instruction-In), essentially shows that (1) utilizing lower-level caches (e.g., LLC) for Prime+Probe attacks via data conflicts arising from instruction fetching represents an expansion of prior attack methodologies, potentially circumventing previously implemented mitigation tactics, and (2) exploiting data-instruction conflicts can facilitate Spectre attacks that bypass proposed speculative defenses. Following the acquisition of a secret, fetching secret-dependent instructions may induce cache conflicts in the shared (between data and instructions) caches, leading to evictions

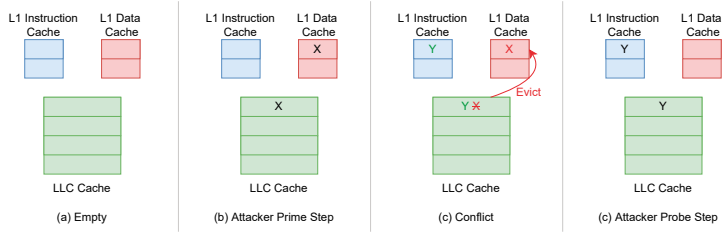


Figure IV.1. Steps of conflicting data and instructions: (a) The cache is empty. (b) Attacker primes the L1D cache. (c) Conflict with the previously cached data in the LLC. The instruction replace s the data in the LLC and evicts the data from the L1D cache. (d) Attacker probes the data in the L1D cache.

in the L1D cache and creating observable timing differences. In essence, despite Delay-on-Miss protections, it remains feasible to expose a secret through data cache based methods.

It was known that attacks, as applied with data, can also be applied with instructions and hence similarly leak information. What DOIN! shows for the first time, is that instead of having only data attacks, or only instruction fetch attacks, an attack can be performed that combines both data and instructions. Hence, if a defense mechanism claims that it protects the data cache by protecting data, DOIN! can still leak data information from the data cache by using instructions. Essentially, Paper IV highlights the need to *always protect both caches at the same time*.

Such defenses that focus on protecting against leakage occurring explicitly from data caches (e.g., Delay-on-Miss delays data misses in the L1 data cache), are shown to be vulnerable and Paper IV demonstrates an attack that breaks such defenses.

The following listing presents the pseudo-code of a potential DOIN! attack capable of surpassing DoM defense.

```
prime_L1D_set();
if(mispredict){
    if(secret){
        // instruction address PC1, PC2, ... PCn conflict
        // in lower level caches with data addresses of the
        // primed L1 data cache (prime_L1D_set())
        PC1
        PC2
        ...
        PCn
    }
}
// if secret == 1 the PC addresses will have evicted primed data
// addresses from lower-level caches (e.g., LLC), and because of
// inclusive behavior, from L1 data cache as well.
// Hence, misses should be observed during the execution of the
// probe function.
probe_L1D_set();
```

In step (1) DOIN! primes a cache set in the data cache non-speculatively. These data that appear in the L1 data cache, will also appear in lower-level caches (e.g., LLC) due to inclusive behavior. Then in step (2), under speculation, DOIN! uses a secret-dependent branch instruction, to follow either a path that fetches instructions that their addresses (program counter) are highly likely to conflict with the primed data that has been inserted in the lower level caches, or a path that fetches no instructions (essentially does nothing). Instructions that will conflict and evict data from lower-level caches, due to inclusive behavior, will also force eviction of the same data from the data cache (this is based purely on inclusive behavior). Hence, without actually accessing any data as transmitters, DOIN! creates observable timing differences in the data cache just by fetching instructions on a secret-dependent path. Such an attack can surpass defenses similar to DoM that focus on not exposing the secret value through the data cache. *In fact,* DOIN! *requires zero data accesses under speculation to leak the value of the secret.*

Paper IV also proposes a simple mitigation strategy, by applying the delay-on-miss technique not only on data, but also, on instructions. Since CPU benchmarks, such as the one in the SPEC2006 benchmark suite, experience no major instruction misses, this mitigation strategy introduces negligible performance degradation, while protecting against attacks that leverage both instructions and data that conflict in the memory hierarchy.

This technique could be considered conservative for a different set of benchmarks, such as datacenter or server workloads, that are known for their large footprint and exhibit high instruction cache miss rates. However, those type of workloads typically run on processors with non-inclusive or exclusive cache hierarchies and thus less relevant to our case.

Nevertheless, several improvements can be made to improve performance on workloads that suffer from more instruction misses. An instruction miss in the first-level cache does not always correspond to a miss and an eviction in the second-level cache since the instruction can also hit in that level. In such scenario, the attack is unsuccessful since no conflict miss can take place. The defense can be equally effective by delaying instructions that miss on the second-level cache, eliminating the need of front-end stalls due to first-level instruction misses. Furthermore, such a delaying strategy takes place under an active speculation window, for example while there exists an older branch in the pipeline. The speculation's outcome can be known much earlier [32, 35] (e.g., when the operands of a branch instruction are ready), guaranteeing that speculation will eventually commit and will not squash the subsequent instructions. In such a case, the front-end can continue as normal without delaying further instructions, even when a branch is still in the pipeline.

IV.2 Paper V - Reorder Buffer Contention: A Forward Speculative Interference Attack for Speculation Invariant Instructions

Paper V describes an attack that is able to break defenses like InvarSpec.

The insight
Defense optimizations such as InvarSpec [43] enhance the performance of defenses such as Delay-on-Miss by enabling the execution of delayed speculative loads that are bound-to-commit regardless the outcome of speculation. Reorder Buffer Contention has the insight that since Delay-on-Miss allows the use of the secret when it hits in the first level cache, those bound-to-commit instructions can still act as transmitters implicitly if a way is found to control their execution depending on the secret value. Hence, by exploiting the reorder buffer in a secret-dependent manner, those instructions enabled by InvarSpec as safe, can now be executed to hint the value of the secret.

Overview of REORDER BUFFER CONTENTION
Paper V presents a speculative attack that is able to extract secret information by influencing the execution of a younger (later in program order) transmitter instruction.

As the name suggests, REORDER BUFFER CONTENTION adds contention to the reorder buffer (ROB) in a secret-dependent manner, under speculation. This added contention influences the execution of a younger, in program order, instruction (a load transmitter instruction), by pushing it in or out the ROB on demand. Hence, this load transmitter will execute under speculation if no ROB contention has been added, otherwise it will never execute under speculation. Once the speculation window finishes, executing and measuring the access of the load transmitter will reveal the path that was taken and consequently the value of the secret. The success of such an attack is demonstrated against InvarSpec [43] framework, since InvarSpec allows for both the load of the secret and the execution of the transmitter instruction.

While the contention can be added by a secret-dependent branch that follows a path of a spurious number of instructions, Paper V adds contention by executing a single instruction that has the ability to unroll as a micro-op loop in the front-end. Such an instruction appears in x86 ISA, is called `REP`, and creates a single-instruction loop, with the value stored in the `ECX` register acting as the loop counter, unrolling at *decode time* [1]. Thus, ROB occupancy becomes a function of `ECX`.

The following listing shows a potential attack code pattern.

The secret is left-shifted (by ten places in this code) to create a large loop iteration number (note that if the secret is zero the left-shift has no effect). Then the secret is loaded into the `ECX` register and acts as the repetition factor for the `REP movs` instruction. The transmitter instruction either executes

```
1  if(mispredict){
2
3   secret = secret << 10; // Repetition factor
4
5   // Pass secret to ECX and execute rep
6   asm("movl␣%0,␣%%ecx" : : "c" (secret));
7   asm("rep␣movsb");
8  }
9  else{
10     // Correct path of execution
11     measure(probe[0]);
12 }
13 transmitter = probe[0];
```

during the speculation window (if secret is 0) after the branch recovergence point (line 13), or does not execute since the loop unrolls in the front-end adding contention in the pipeline for the duration of the speculation window. When mispeculation is verified the execution continues from the correct path of execution (line 9). The cache access latency of the transmitter instruction is measured in order to infer the value of the secret. If the access hits in the cache then the transmitter (line 13) executed during the speculation, if it misses then it did not execute during speculation.

V. Conclusion

This thesis tries to address two challenges in the speculative side-channel realm, performance and security. At first, it proposes three works, namely Janus, ReCon, and Doppelganger Loads, to reclaim lost performance introduced by previous defense mechanisms, such as Non-speculative Data Access (NDA), Speculative Taint Tracking (STT) and Delay-on-Miss (DoM). Secondly, it tackles the security issues of existing mechanisms and proposes novel attack ideas, by introducing DOIN! and Reorder Buffer Contention attacks (and an attack that is included in Paper I).

Targeting performance, Paper I highlights the complementary nature of NDA and DoM. Janus is a reinforcement learning agent that is employed in a processor design that utilizes both NDA and DoM mechanisms and dynamically adapts its defense strategy based on performance feedback. Essentially, Janus switches between either NDA or DoM but they never run concurrently and never under the same speculation window, and evaluates the program's behavior to identify which mechanism fits best the running workload. The architecture is characterized by simplicity since it is based on easy-to-implement defense mechanisms and a lightweight agent of just 16 bytes. Moreover, even though Paper I applies the two mechanisms independently, it includes a study on potential vulnerabilities that would occur if both mechanisms operated under the same speculation window. The study shows that when defenses like DoM and NDA operate under the same speculation window, or combined to work together in one processor design, allow both the use of the secret information and the creation of observable timing differences. Such behavior can be harmful, and Janus proves it by demonstrating an attack scenario on prior work.

Paper II uses non-speculative information leakage to enable the execution of speculative load instructions that perform memory accesses to non-secret information. Paper II makes the observation that 60% of all the non-speculative information leakage comes from direct load-load dependences (e.g., pointer dereferencing), introducing a framework named ReCon that takes advantage of this characteristic. Based on this observation, ReCon proposes a simple and efficient design to capture the information leakage arising from direct load-load papers using a single table structure, and tags the memory locations that are exposed by these load pairs in the cache hierarchy as public or non-public. Upon speculative load accesses, the underlying mechanism leverages this information to apply restrictions (accessing non-public memory locations) or continue without restrictions (accessing public memory locations).

Paper III introduces address prediction to enable more Memory-Level Parallelism (MLP) on speculative defenses that restrict the execution of speculative load instructions and as a consequence limits the application's MLP. Doppelganger Loads execute load-doubles for every issued load instruction by predicting their address, if the predictor is confident in the prediction. The load executes with a predicted address, but never actually propagates its value or performs any other action, until the underlying mechanism allows it to do so. Essentially, Doppelganger Loads prepares the speculative restricted loads based on a prediction, before their address is even generated. If the predicted address was correct, then the doppelganger has started cycles earlier. If the predicted address was not correct, then the actual load continues as normal by doing the memory access when the underlying mechanism allows it to. Doppelganger Loads is a lightweight solution, as it can utilize existing structures in the architecture (e.g., prefetchers) to perform predictions.

On a different direction, Paper IV tries to unveil vulnerabilities in existing defenses. Exploiting inclusive caches, Paper IV first shows that Prime+Probe attacks can be done by mixing data and instructions in the lower level caches (e.g., LLC), demonstrating an attack named Data-Out Instruction-In (DOIN!). With such an observation, Paper IV proposes a speculative DOIN! attack which can surpass speculative defenses that are built to protect against leakage occurring from the data cache. DOIN! leaks information without actually using data. The core idea of the attack is that an attacker can prime the data cache legally and non-speculatively (and thus because of inclusive caches all primed data in L1 data cache will appear in lower level caches such as LLC as well), and then leverage secret-dependent instruction fetches that will conflict with the data in lower level caches (e.g., LLC), forcing evictions from the lower level cache (and also from the data cache since caches are inclusive), creating observable timing differences in the L1 data cache. DOIN! essentially shows that protecting L1 data caches from leaking information, not only enough data protection is required, but also protecting instructions is mandatory. Finally, Paper IV proposes a simple mitigation strategy.

Paper V, in a similar concept as Paper IV, demonstrates an attack scenario that adds secret-dependent contention to the reorder buffer, essentially pushing in or out an independent transmitter instruction. Depending on if the transmitter instruction executes or not, the attacker can infer if contention was added, and hence the value of the secret since added contention is secret-dependent. Paper IV performs the attack using a single instruction that can unroll into multiple micro-ops in the front end. Basically, the amount of unrolling is dependent on the secret value.

Since the disclosure of Spectre attacks in 2018, the community has made tremendous steps forward in tackling the problem directly from hardware. Yet, defending against speculative attacks in a comprehensive, efficient, and simple manner is still an open problem. I hope, that this thesis and the ideas intro-

duced in it, will inspire the community for more novel ideas targeting faster and safer speculation.

Svensk Sammanfattning

Under senare år har säkerheten inom datorarkitektur fått betydande uppmärksamhet, särskilt sedan avslöjandet av Spectre- och Meltdown -attackerna 2018. Dessa attacker utnyttjar spekulativ exekvering, en funktion i centrala processorer (CPU), för att läcka data som normalt är otillgängliga under programkörning. För att möta denna utmaning har ett flertal försvar föreslagits, dock på bekostnad av betydande prestandaförsämring och designkomplexitet. Denna avhandling närmar sig problemet från två olika perspektiv. För det första strävar den efter att återvinna förlorad prestanda genom att implementera nya tekniker på etablerade toppmoderna mekanismer. För det andra bevisar den att vissa av de befintliga mekanismerna fortfarande är sårbara, genom att demonstrera nya attackscenarier på dem.

Ut-av-ordning exekvering syftar på moderna processorers förmåga att exekvera instruktioner i en ordning som inte fokuserar på när en instruktion kommer in i pipelinen (i ordning) utan när den är redo att exekvera. Med ut-av-ordning exekvering kan yngre instruktioner som är redo att exekvera kringgå äldre instruktioner som inte är redo att exekvera. Som ett resultat, när instruktioner som behöver fatta ett beslut påträffas (t.ex. villkorliga förgreningsinstruktioner), eftersom inte alla föregående instruktioner har exekverats och därmed är villkorstillståndet okänt, blir det utmanande att bestämma den exakta exekveringsvägen efter dessa specifika instruktioner. Istället för att vänta tills alla tidigare instruktioner har slutfört exekveringen, förlitar sig processorer på olika heuristik och tidigare exekveringsmönster för att göra välgrundade förutsägelser och fortsätta programkörningen. Denna specifika funktion, där instruktioner spekuleras och exekveras längs vägar som kan innebära osäkerhet angående korrektheten, beskriver i huvudsak termen *spekulativ exekvering*. Spekulativ exekvering är en av de viktigaste funktionerna i moderna datorarkitekturer som dramatiskt förbättrar den totala systemprestandan, eftersom den tillåter processorn att fortsätta exekveringen, utan fördröjningar, vanligtvis med hög korrekthet i förutsägelserna.

Säkert är det så att processorn i vissa fall gör felaktiga förutsägelser och exekverar instruktioner längs fel väg. Processorer avstår från att commitera eller pensionera spekulativt exekverade instruktioner; istället kvarstår dessa instruktioner i pipelinen tills processorn bestämmer det faktiska utfallet av den förutsagda vägen. Om förutsägelsen är korrekt commitas instruktionerna, och exekveringen fortsätter som normalt. Å andra sidan, om processorn exekverar längs fel väg, kastar den bort alla spekulativa ändringar och återställer alla arkitektoniska tillstånd som om spekulativ exekvering aldrig inträffat. Det

arkitektoniska tillståndet avser processorns externt synliga tillstånd så som det uppfattas av programvaran eller programmeraren, såsom register, som är direkt tillgängliga och manipuleras av programvaran som körs på processorn. Det mikroarkitektoniska tillståndet avser processorns interna tillstånd som inte är direkt synligt för programvara, såsom innehållet i datacachen. Vid en felaktig förutsägelse, även om det arkitektoniska tillståndet återställs, kvarstår de mikroarkitektoniska förändringarna och lämnar spår som kan utnyttjas av potentiella angripare. Det mikroarkitektoniska tillståndet är inte tillgängligt för programvarulagret, typiskt behöver en angripare bilda en sidokanal för att läcka sådan information, till exempel en tidsmässig sidokanal (t.ex. mätning av tidpunkten för en minnesåtkomst). Genom att bilda en tidsmässig sidokanal kan en angripare extrahera ledtrådar om programbeteendet (bestämma minnesåtkomstmönstret för ett offerprogram genom att övervaka vilka adresser offret kommer åt), eller till och med specifika datavärden (genom att avreferera minnesplatser och skapa observerbara tidsskillnader baserade på värden).

Genom att kombinera spekulativ exekvering och sidokanaler uppstår spekulativa sidokanaler. I en spekulativ sidokanalsattack kommer en hemlighet åt på ett felaktigt sätt spekulativt (under felaktig vägexekvering) och läcks sedan genom att passera den till en sändarinstruktion medan den fortfarande är under spekulation. Även om felaktig förutsägelse identifieras och det arkitektoniska tillståndet återställs, kvarstår det mikroarkitektoniska tillståndet och kan utnyttjas. I en senare fas bildar en angripare en sidokanal (som den tidsmässiga sidokanalen som diskuterats ovan) och läcker hemlighetens värde, till exempel genom att mäta tiden för en specifik cachelinjeåtkomst.

Som framgår av föregående stycken ligger kärnan i spekulativa sidokanalsattacker i spekulativa laddningsinstruktioner. Under spekulativ exekvering har processorn ett “fritt pass i minnet”. Ett sätt att hantera spekulativa attacker vore att eliminera spekulativ exekvering, men det skulle dra tillbaka prestandaförbättringar under de senaste decennierna. En annan naiv lösning vore att fördröja exekveringen av alla spekulativa laddningsinstruktioner, men detta skulle också medföra extrema prestandakostnader. Därför introducerades mer sofistikerade försvar, som angriper problemet från helt olika synvinklar. Dessa försvar utforskar tekniker såsom buffring, begränsad spridning, selektiv fördröjning, taint tracking, och många andra. Även om dessa tekniker lyckas mildra prestandakostnader i varierande grad, misslyckas de typiskt med att eliminera dem helt, introducerar betydande design- och implementeringskomplexitet, och når ibland inte fullt ut i att åtgärda säkerhetssårbarheter. Denna avhandling, och uppsatserna I-V, tacklar exakt alla dessa problem.

Uppsatserna I-III fokuserar på att återvinna förlorad prestanda genom att föreslå enkla och praktiska designer. Uppsats I introducerar Janus, ett tillvägagångssätt som uttryckligen fokuserar på att minska implementeringskomplexiteten samtidigt som förlorad prestanda återvinns jämfört med tidigare föreslagna lösningar. Janus använder en lättviktig förstärkningsinlärningsagent för att dynamiskt anpassa sig till programbeteendet och välja den mest op-

timala, prestandamässigt sett, försvarsmekanismen att köra som den underliggande mekanismen. Trots att förstärkningsinlärning är kopplad till implementeringskomplexitet, använder Janus den enklaste och lättaste formen av förstärkningsinlärning, vilket introducerar försumbara designkostnader. Janus väljer mellan två av de enklaste försvarsidéerna som kan existera tillsammans i en enda processordesign och visar att de också kan vara komplementära. Uppsatsen introducerar också ett attackkoncept baserat på hur cacheträffar och -missar kan påverka beteendet hos kommande minnesinstruktioner. Uppsats II introducerar ReCon, en optimering för befintliga spekulativa sidokanalsförsvar som utnyttjar icke-spekulativ informationsläckage (läckage som sker under programmets normala exekvering) för att förbättra prestandan hos den underliggande försvarsmekanismen som begränsar spekulation. ReCon spårar icke-spekulativt informationsläckage, markerar data i minneshierarkin som avslöjade (offentlig information) och dolda (potentiella hemligheter), och använder denna information för att möjliggöra mer parallellitet på minnesnivå även under spekulativa begränsningar.

Uppsats III introducerar Doppelganger Loads, också en optimering för befintliga spekulativa sidokanalsförsvar. Doppelganger Loads utnyttjar adressprediktion för att möjliggöra exekvering av spekulativa laddningar oberoende av hemligheter, vilket minimerar påverkan på prestanda orsakad av begränsad spekulativ exekvering.

Uppsatserna IV och V fokuserar på att åtgärda säkerhetssårbarheter i föreslagna sidokanalsförsvar. Uppsats IV introducerar DOIN!, en attack som utnyttjar inkluderande cachebeteende för att få data och instruktioner att interferera i de lägre nivåernas cacheminnen och skapa observerbara skillnader i datacachen utan att faktiskt använda data, utan instruktioner — eftersom användning av data är vad specifika försvar begränsar dig från. Därför, även om ett försvar inte tillåter exekvering av laddningsinstruktioner för att överföra information i datacachen, visar DOIN! att du kan hämta vilken typ av instruktion som helst och fortfarande skapa observerbara tidsskillnader i datacachen. Uppsatsen föreslår också en naiv lösning på en sådan attack genom att inkludera instruktioner i skyddsschemat. Slutligen introducerar Uppsats V Reorder Buffer Contention, en attack som fyller omordningsbufferten på begäran baserat på ett hemligt värde och skjuter in eller ut, kontrollerar dess exekvering, en sändarinstruktion. Reorder Buffer Contention kan skapa observerbara tidsskillnader och läcka information genom att komma åt en helt oberoende, i förhållande till kontroll- och dataflödesvägen, instruktion, om det underliggande försvaret tillåter dig att göra det. Dessutom introduceras i Uppsats I en attack som läcker information genom att bilda en kanal baserad på hemlighetsberoende cacheträffar och -missar, utan att använda någon annan explicit instruktion (som till exempel en kontrollinstruktion).

Denna avhandling presenterar totalt fem arbeten. Tre av dem, nämligen Janus, ReCon och Doppelganger Loads, fokuserar på att *förbättra försvar* när det gäller prestanda och/eller komplexitet, och två av dem, nämligen DOIN!

och Reorder Buffer Contention (och en attack inom Janus-arbetet), fokuserar på att *avslöja sårbarheter*.

VI. References

[1] Fog Agner. Instruction tables. `https://www.agner.org/optimize/instruction_tables.pdf`, May 2021.

[2] Pavlos Aimoniotis and Stefanos Kaxiras. Janus: A simple and efficient speculative defense using reinforcement learning. In *Under submission*, 2024.

[3] Pavlos Aimoniotis, Amund Bergland Kvalsvik, Xiaoyue Chen, Magnus Själander, and Stefanos Kaxiras. Recon: Efficient detection, management, and use of non-speculative information leakage. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, pages 828–842, 2023.

[4] Pavlos Aimoniotis, Amund Bergland Kvalsvik, Magnus Själander, and Stefanos Kaxiras. Data-out instruction-in (DOIN!): Leveraging inclusive caches to attack speculative delay schemes. In *Proceedings of the IEEE International Symposium on Secure and Private Execution Environment Design*, pages 49–60, September 2022.

[5] Pavlos Aimoniotis, Christos Sakalis, Magnus Själander, and Stefanos Kaxiras. Reorder buffer contention: A forward speculative interference attack for speculation invariant instructions. *IEEE Computer Architecture Letters*, 20:162–165, July 2021.

[6] Sam Ainsworth. GhostMinion: A strictness-ordered cache system for spectre mitigation. In *Proceedings of the IEEE/ACM International Symposium on Microarchitecture*, page 592–606. Association for Computing Machinery, October 2021.

[7] Sam Ainsworth and Timothy M. Jones. MuonTrap: Preventing cross-domain Spectre-like attacks by capturing speculative state. In *Proceedings of the International Symposium on Computer Architecture*, pages 132–144, May 2020.

[8] Mohammad Behnia, Prateek Sahu, Riccardo Paccagnella, Jiyong Yu, Zirui Neil Zhao, Xiang Zou, Thomas Unterluggauer, Josep Torrellas, Carlos Rozas, Adam Morrison, Frank Mckeen, Fangfei Liu, Ron Gabor, Christopher W. Fletcher, Abhishek Basak, and Alaa Alameldeen. Speculative interference attacks: breaking invisible speculation schemes. In *Proceedings of the Architectural Support for Programming Languages and Operating Systems*, page 1046–1060. Association for Computing Machinery, April 2021.

[9] Daniel J Bernstein. Cache-timing attacks on aes. 2005.

[10] Atri Bhattacharyya, Alexandra Sandulescu, Matthias Neugschwandtner, Alessandro Sorniotti, Babak Falsafi, Mathias Payer, and Anil Kurmus. SMoTherSpectre: Exploiting speculative execution through port contention. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*, page 785–800. Association for Computing Machinery, November 2019.

[11] Rutvik Choudhary, Jiyong Yu, Christopher Fletcher, and Adam Morrison. Speculative privacy tracking (SPT): Leaking information from speculative execution without compromising privacy. In *Proceedings of the IEEE/ACM International Symposium on Microarchitecture*, page 607–622. Association for Computing Machinery, October 2021.

[12] Shuwen Deng, Bowen Huang, and Jakub Szefer. Leaky frontends: Security vulnerabilities in processor frontends. In *Proceedings of the International Symposium High-Performance Computer Architecture*, pages 53–66, April 2022.

[13] Daniel Gruss, Clémentine Maurice, Klaus Wagner, and Stefan Mangard. Flush+Flush: A fast and stealthy cache attack. In *Proceedings of the International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*, pages 279–299. Springer International Publishing, 2016.

[14] Daniel Gruss, Raphael Spreitzer, and Stefan Mangard. Cache template attacks: automating attacks on inclusive last-level caches. In *Proceedings of the USENIX Security Symposium*, page 897–912. USENIX Association, August 2015.

[15] jannh@google.com. Issue 1528: speculative execution, variant 4: speculative store bypass - project-zero, February 2018.

[16] Hai Jin, Zhuo He, and Weizhong Qiang. SpecTerminator: Blocking speculative side channels based on instruction classes on risc-v. *ACM Transactions on Architecture and Code Optimization*, page 3566053, November 2022.

[17] Khaled N. Khasawneh, Esmaeil Mohammadian Koruyeh, Chengyu Song, Dmitry Evtyushkin, Dmitry Ponomarev, and Nael Abu-Ghazaleh. SafeSpec: Banishing the spectre of a meltdown with leakage-free speculation. In *Proceedings of the ACM/IEEE Design Automation Conference*, pages 1–6, June 2019.

[18] Joonsung Kim, Hamin Jang, Hunjun Lee, Seungho Lee, and Jangwoo Kim. UC-Check: Characterizing micro-operation caches in x86 processors and implications in security and performance. In *Proceedings of the IEEE/ACM International Symposium on Microarchitecture*, page 550–564. Association for Computing Machinery, October 2021.

[19] Paul Kocher, Jann Horn, Anders Fogh, Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg, Moritz Lipp, Stefan Mangard, Thomas Prescher, Michael Schwarz, and Yuval Yarom. Spectre attacks: Exploiting speculative execution. In *Proceedings of the IEEE Symposium on Security and Privacy*, pages 1–19, May 2019.

[20] Amund Bergland Kvalsvik, Pavlos Aimoniotis, Stefanos Kaxiras, and Magnus Själander. Doppelganger loads: A safe, complexity-effective optimization for secure speculation schemes. In *Proceedings of the International Symposium on Computer Architecture*, pages 1–13. Association for Computing Machinery, June 2023.

[21] Mengming Li, Chenlu Miao, Yilong Yang, and Kai Bu. unXpec: Breaking undo-based safe speculation. In *Proceedings of the International Symposium High-Performance Computer Architecture*, pages 98–112, April 2022.

[22] Peinan Li, Lutan Zhao, Rui Hou, Lixin Zhang, and Dan Meng. Conditional speculation: An effective approach to safeguard out-of-order execution against

Spectre attacks. In *Proceedings of the International Symposium High-Performance Computer Architecture*, pages 264–276. IEEE Computer Society, February 2019.

[23] Moritz Lipp, Michael Schwarz, Daniel Gruss, Thomas Prescher, Werner Haas, Anders Fogh, Jann Horn, Stefan Mangard, Paul Kocher, Daniel Genkin, Yuval Yarom, and Mike Hamburg. Meltdown: Reading kernel memory from user space. In *Proceedings of the USENIX Security Symposium*, 2018.

[24] Fangfei Liu, Yuval Yarom, Qian Ge, Gernot Heiser, and Ruby B. Lee. Last-level cache side-channel attacks are practical. In *Proceedings of the IEEE Symposium on Security and Privacy*, pages 605–622, May 2015.

[25] Kevin Loughlin, Ian Neal, Jiacheng Ma, Elisa Tsai, Ofir Weisse, Satish Narayanasamy, and Baris Kasikci. DOLMA: Securing speculation with the principle of transient non-observability. In *Proceedings of the USENIX Security Symposium*, 2021.

[26] Ross Mcilroy, Jaroslav Sevcik, Tobias Tebbi, Ben L. Titzer, and Toon Verwaest. Spectre is here to stay: An analysis of side-channels and speculative execution. February 2019.

[27] Dag Arne Osvik, Adi Shamir, and Eran Tromer. Cache attacks and countermeasures: The case of AES. In *Proceedings of the RSA Conference*, pages 1–20. Springer, 2006.

[28] Joseph Ravichandran, Weon Taek Na, Jay Lang, and Mengjia Yan. PACMAN: attacking arm pointer authentication with speculative execution. In *Proceedings of the International Symposium on Computer Architecture*, page 685–698. Association for Computing Machinery, June 2022.

[29] Xida Ren, Logan Moody, Mohammadkazem Taram, Matthew Jordan, Dean M Tullsen, and Ashish Venkat. I see dead µops: Leaking secrets via Intel/AMD micro-op caches. In *Proceedings of the International Symposium on Computer Architecture*, page 14, June 2021.

[30] Gururaj Saileshwar and Moinuddin K. Qureshi. CleanupSpec: An "undo" approach to safe speculation. In *Proceedings of the IEEE/ACM International Symposium on Microarchitecture*, page 73–86. Association for Computing Machinery, October 2019.

[31] Christos Sakalis, Mehdi Alipour, Alberto Ros, Alexandra Jimborean, Stefanos Kaxiras, and Magnus Själander. Ghost loads: What is the cost of invisible speculation? In *Proceedings of the ACM International Conference on Computing Frontiers*, page 153–163. Association for Computing Machinery, April 2019.

[32] Christos Sakalis, Stefanos Kaxiras, Alberto Ros, Alexandra Jimborean, and Magnus Själander. Efficient invisible speculative execution through selective delay and value prediction. In *Proceedings of the International Symposium on Computer Architecture*, pages 723–735, June 2019.

[33] Christos Sakalis, Magnus Själander, and Stefanos Kaxiras. Seeds of SEED: Preventing priority inversion in instruction scheduling to disrupt speculative interference. In *Proceedings of the IEEE International Symposium on Secure and Private Execution Environment Design*, pages 101–107, September 2021.

[34] G Edward Suh, Jaewook Lee, and Srinivas Devadas. Secure program execution via dynamic information flow tracking. *ACM SIGPLAN Notices*, 39:85–96,

2004.

[35] Kim-Anh Tran, Christos Sakalis, Magnus Själander, Alberto Ros, Stefanos Kaxiras, and Alexandra Jimborean. Clearing the shadows: Recovering lost performance for invisible speculative execution through HW/SW co-design. In *Proceedings of the International Conference on Parallel Architectural and Compilation Techniques*, page 241–254. Association for Computing Machinery, September 2020.

[36] Ofir Weisse, Ian Neal, Kevin Loughlin, Thomas F. Wenisch, and Baris Kasikci. NDA: Preventing speculative execution attacks at their source. In *Proceedings of the IEEE/ACM International Symposium on Microarchitecture*, page 572–586. Association for Computing Machinery, October 2019.

[37] Johannes Wikner, Daniël Trujillo, and Kaveh Razavi. Phantom: Exploiting decoder-detectable mispredictions. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, pages 49–61, 2023.

[38] Mengjia Yan, Jiho Choi, Dimitrios Skarlatos, Adam Morrison, Christopher Fletcher, and Josep Torrellas. InvisiSpec: Making speculative execution invisible in the cache hierarchy. In *Proceedings of the IEEE/ACM International Symposium on Microarchitecture*, pages 428–441, October 2018.

[39] Yuval Yarom and Katrina Falkner. FLUSH+RELOAD: a high resolution, low noise, L3 cache side-channel attack. In *Proceedings of the USENIX Security Symposium*, page 719–732. USENIX Association, August 2014.

[40] Jiyong Yu, Namrata Mantri, Josep Torrellas, Adam Morrison, and Christopher W. Fletcher. Speculative data-oblivious execution: Mobilizing safe prediction for safe and efficient speculative execution. In *Proceedings of the International Symposium on Computer Architecture*, pages 707–720, May 2020.

[41] Jiyong Yu, Mengjia Yan, Artem Khyzha, Adam Morrison, Josep Torrellas, and Christopher W. Fletcher. Speculative taint tracking (STT): A comprehensive protection for speculatively accessed data. In *Proceedings of the IEEE/ACM International Symposium on Microarchitecture*, page 954–968. Association for Computing Machinery, October 2019.

[42] Zirui Neil Zhao, Houxiang Ji, Adam Morrison, Darko Marinov, and Josep Torrellas. Pinned loads: taming speculative loads in secure processors. In *Proceedings of the Architectural Support for Programming Languages and Operating Systems*, page 314–328. Association for Computing Machinery, February 2022.

[43] Zirui Neil Zhao, Houxiang Ji, Mengjia Yan, Jiyong Yu, Christopher W. Fletcher, Adam Morrison, Darko Marinov, and Josep Torrellas. Speculation invariance (InvarSpec): Faster safe execution through program analysis. In *Proceedings of the IEEE/ACM International Symposium on Microarchitecture*, pages 1138–1152. IEEE Computer Society, October 2020.

Acta Universitatis Upsaliensis

Digital Comprehensive Summaries of Uppsala Dissertations from the Faculty of Science and Technology 2599

Editor: The Dean of the Faculty of Science and Technology

A doctoral dissertation from the Faculty of Science and Technology, Uppsala University, is usually a summary of a number of papers. A few copies of the complete dissertation are kept at major Swedish research libraries, while the summary alone is distributed internationally through the series Digital Comprehensive Summaries of Uppsala Dissertations from the Faculty of Science and Technology. (Prior to January, 2005, the series was published under the title "Comprehensive Summaries of Uppsala Dissertations from the Faculty of Science and Technology".)

Distribution: publications.uu.se
urn:nbn:se:uu:diva-566433

ACTA UNIVERSITATIS
UPSALIENSIS
2025